

FACIAL KEYPOINT DETECTION

Using Deep Learning

Topic 14 — YouTube Faces with Facial Keypoints | **Team Alpha** | Deep Learning

Motivation & Objective

What is Facial Keypoint Detection?

Localize **68 anatomical points** on a face: eye corners, nose tip, mouth edges, jaw contour.

Applications

- Face tracking & expression analysis
- Biometrics & face recognition
- Medical diagnosis (dysmorphic signs)

Task: Regression predict 136 continuous values
(x,y for each of 68 keypoints)

Loss: MSE | $\hat{y} - y^2$

Dataset — YouTube Faces with Facial Keypoints

260K

Total frames

1,245

NPZ files

68

Keypoints/frame

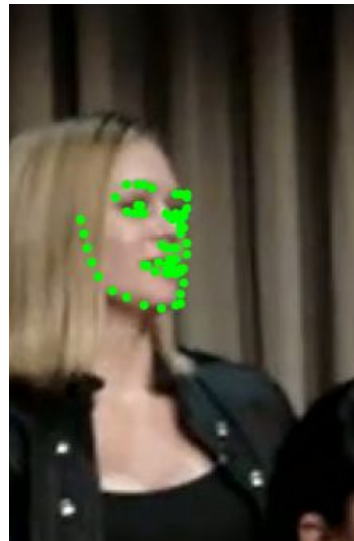
RGB

Color images

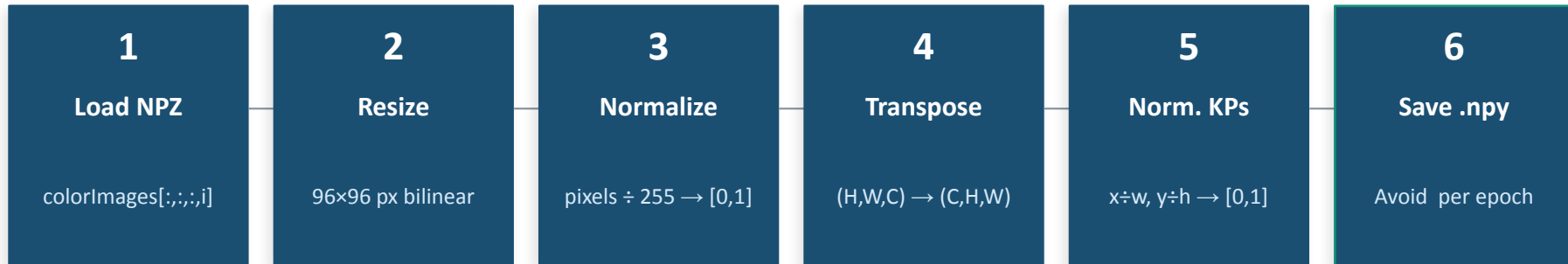
NPZ File Structure

Key	Shape	Description
colorImages	(H, W, 3, frames)	RGB frames — uint8
landmarks2D	(68, 2, frames)	x,y keypoint coords
boundingBox	(4, 2, frames)	Face bounding box

Kaggle: [selfishgene/youtube-faces-with-facial-keypoints](#)



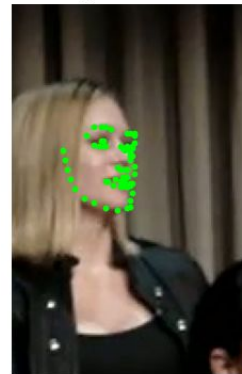
Preprocessing Pipeline



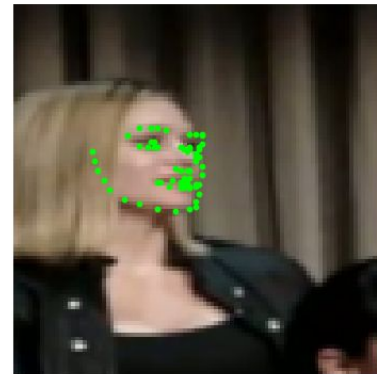
Data Splits

Set	Frames	Source
Train	40,000 (80%)	Random sample (seed=42)
Validation	10,000 (20%)	Random sample (seed=42)
Test	10,000	Unseen pool (210k frames never used)

Original 125x192



Redimensionada 96x96



Models — Baseline CNN vs NaimishNet

Baseline CNN

Input (3, 96, 96)

- Conv2d(3→32) + ReLU + MaxPool → 48×48
- Conv2d(32→64) + ReLU + MaxPool → 24×24
- Conv2d(64→128) + ReLU + MaxPool → 12×12
- Conv2d(128→256) + ReLU + MaxPool → 6×6
- Flatten → FC(9216→1024) → ReLU
- FC(1024→256) → ReLU
- FC(256→136) → Sigmoid

Output: 136 values $\in [0,1]$

NaimishNet (Agarwal et al., 2017)

Feature	Baseline	NaimishNet
Activation	ReLU	ELU
BatchNorm	✗	✓ after each conv
Dropout	✗	0.1 →0.2→0.3→0.4→0.5
Output	Sigmoid [0,1]	Direct regression
Kernels	3×3 all	4,3,2,1 (decreasing)

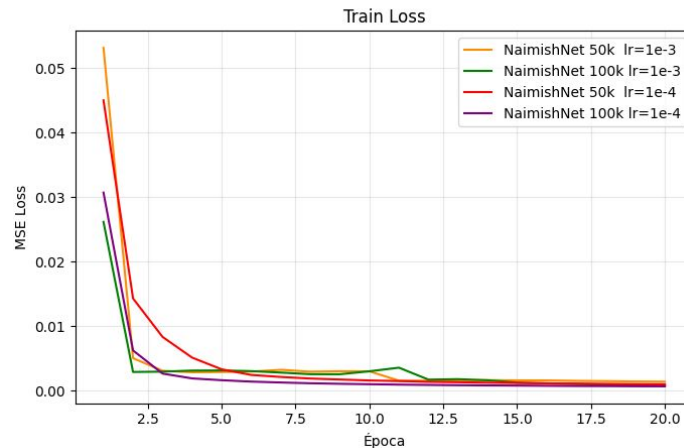
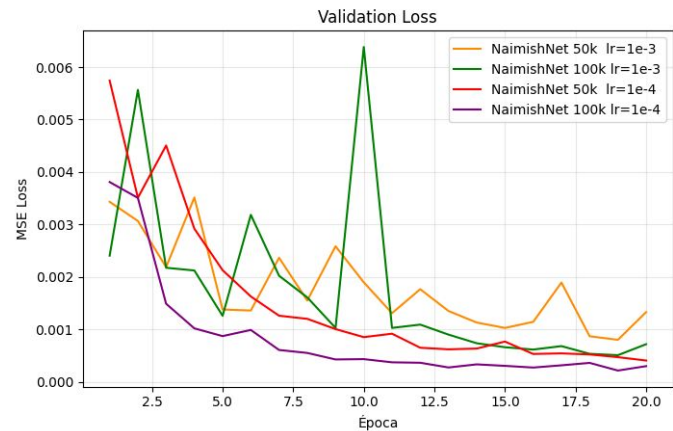
Training Setup & Experiments

Training Configuration

- Optimizer: Adam (Adaptive Moment Estimation)
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=3)
- Loss: MSELoss (L2-loss)
- Epochs: 20 | Mini-batch size: 64

5 Models

Model	Frames	LR
Baseline CNN	50k	1e-3
NaimishNet	50k	1e-3
NaimishNet	100k	1e-3
NaimishNet	50k	1e-4
NaimishNet	100k	1e-4



Results — Quantitative Metrics

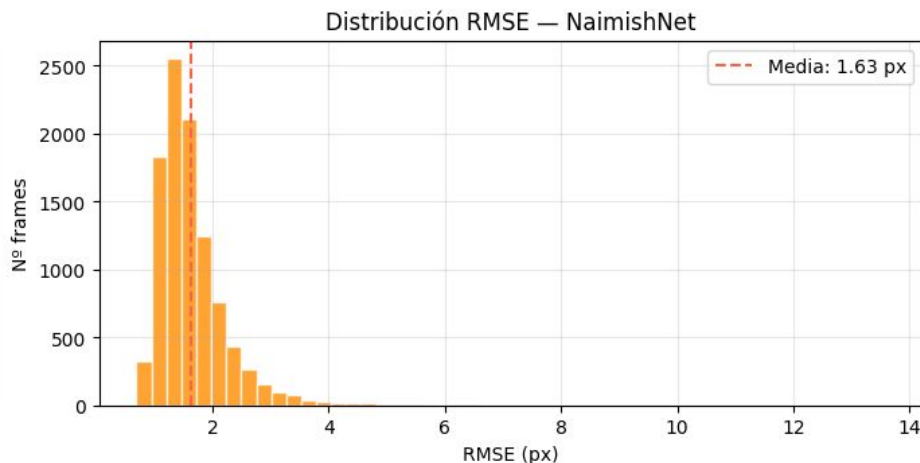
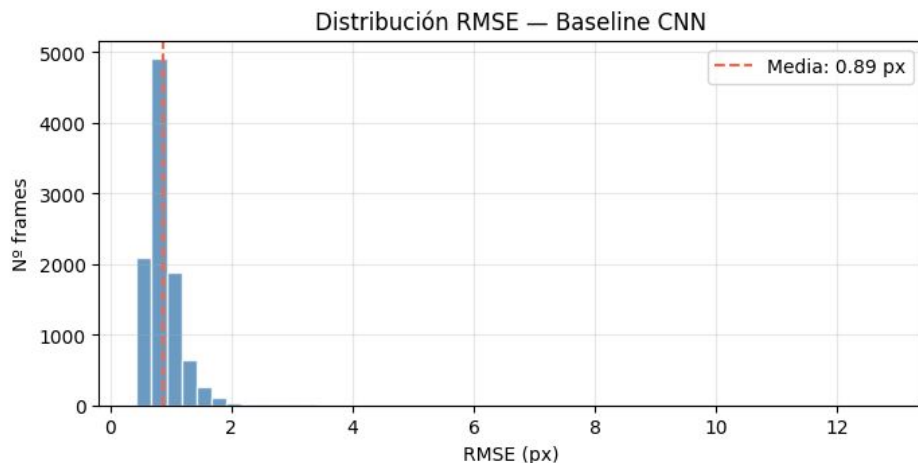
Validation Results — All Models

Model	Frames	LR	Best val MSE	RMSE (px)
Baseline CNN	50k	1e-3	0.000065	0.89 px
NaimishNet	50k	1e-3	0.000802	3.15 px
NaimishNet	100k	1e-3	0.000510	2.36 px
NaimishNet	50k	1e-4	0.000408	2.26 px
NaimishNet	100k	1e-4	0.000217	1.64 px

Generalization — Validation vs Test (unseen data)

Set	Frames	RMSE Baseline	RMSE NaimishNet	Gap
Validation	10,000	0.89 px	1.64 px	—
Test (unseen)	10,000	0.90 px	1.63 px	< 0.6%

Error Analysis — Distribution & Statistics



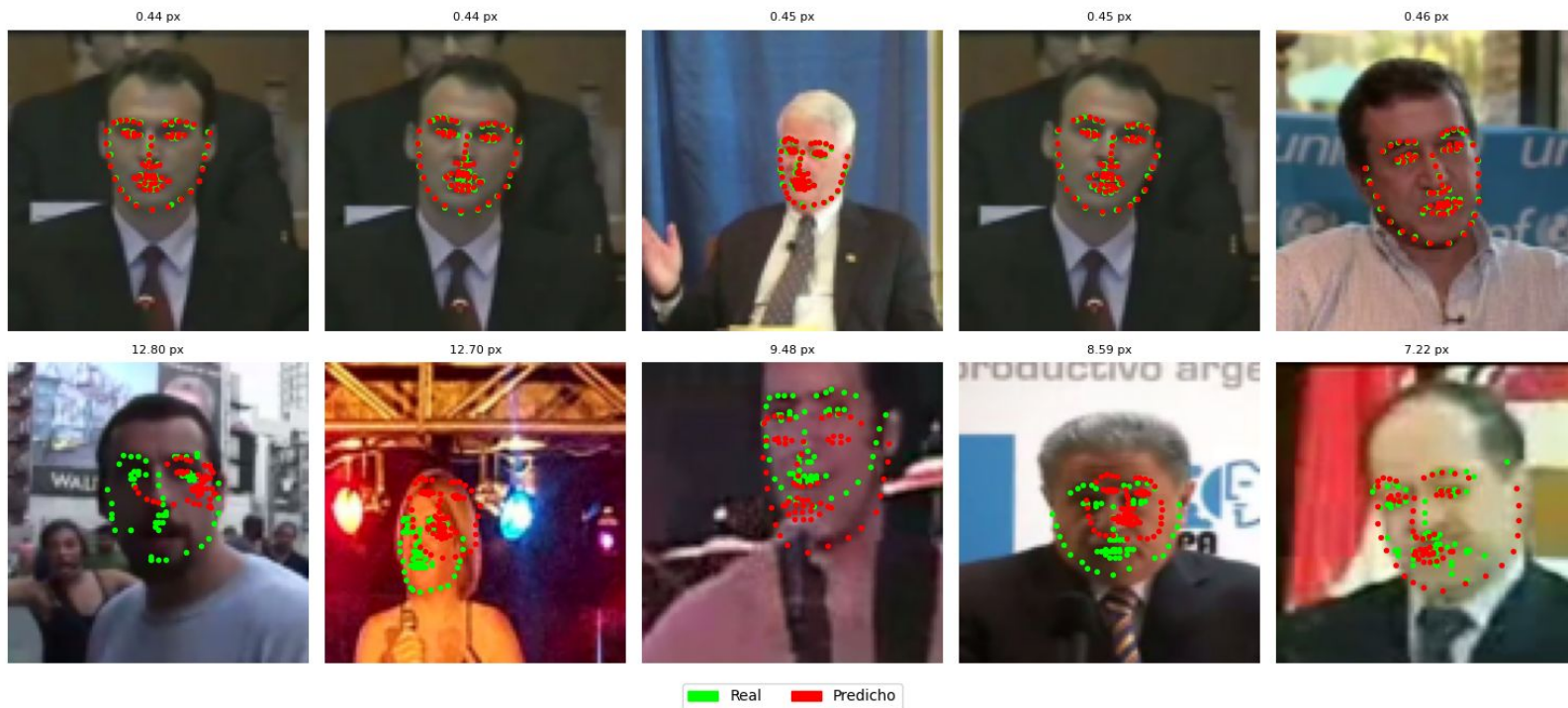
RMSE per-frame statistics on 10,000 test frames

Percentile	Baseline CNN	NaimishNet	Difference
p50 (median)	0.80 px	1.49 px	+0.69
p90	1.21 px	2.32 px	+1.11
p99	1.95 px	3.97 px	+2.02
Frames > 5 px	8 (0.1%)	36 (0.4%)	4x more

Results — Qualitative Analysis

Best frames (low RMSE) vs Worst frames (high RMSE)

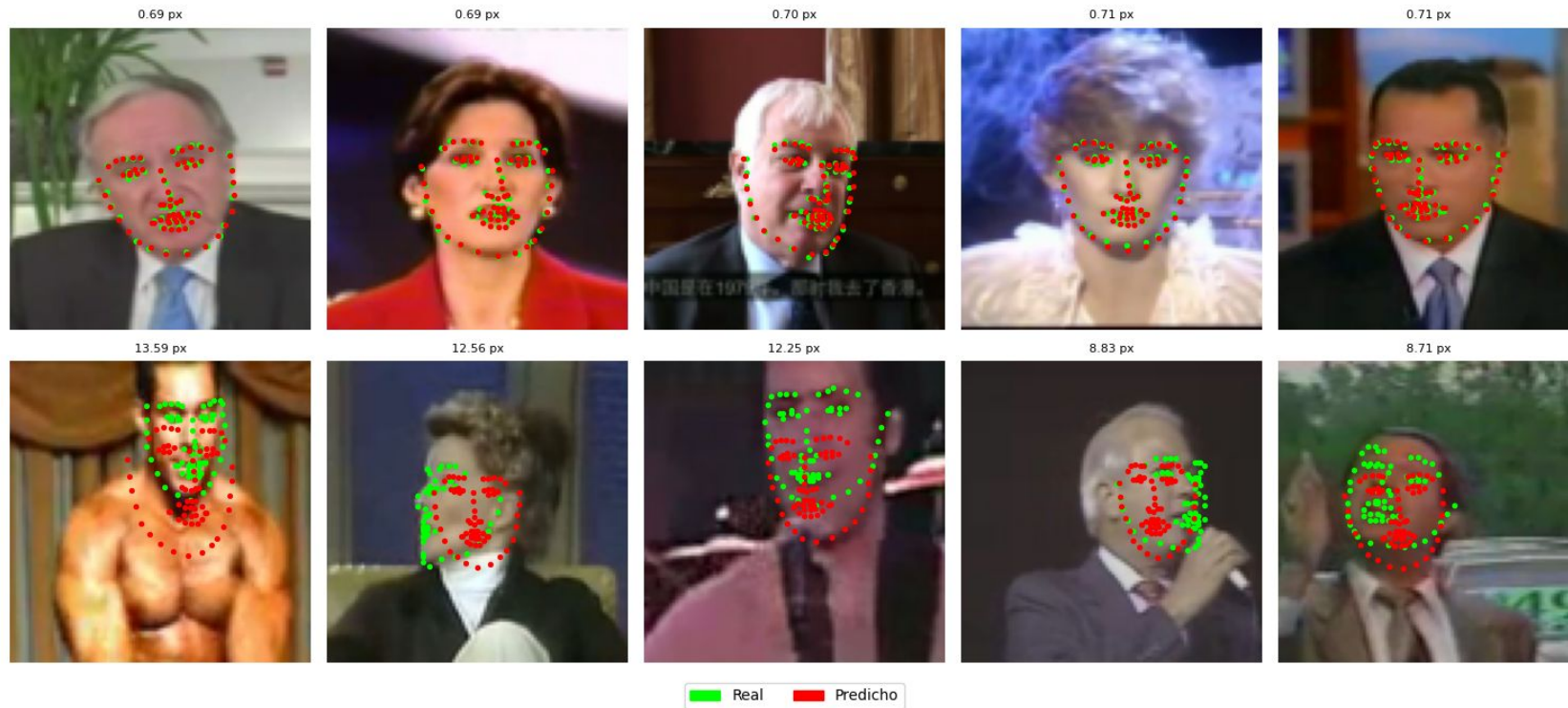
Mejores vs Peores frames — Baseline CNN



Results — Qualitative Analysis

Best frames (low RMSE) vs Worst frames (high RMSE)

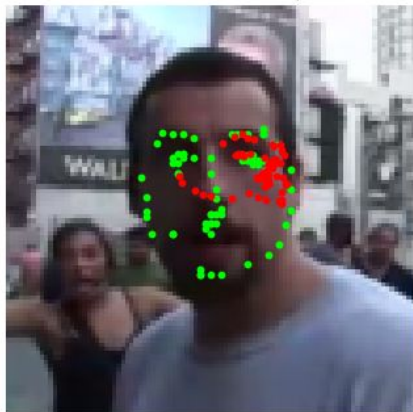
Mejores vs Peores frames — NaimishNet



Baseline vs NaimishNet — Side-by-Side

Top 10 frames difíciles — Baseline vs NaimishNet

Baseline RMSE: 12.80 px



NaimishNet RMSE: 6.17 px



Baseline RMSE: 8.59 px



NaimishNet RMSE: 2.35 px



Discussion

What Worked

- Baseline CNN converges stably with $lr=1e-3$
- RMSE 0.89 px, less than 1 pixel error on 96×96 images
- Excellent generalization: only 0.1% gap val→test

What Didn't Work

- NaimishNet $lr=1e-3$ unstable convergence (loss spikes at epoch 10)
- Excessive Dropout (0.5) for this data volume
- No data augmentation sensitive to lighting & occlusion

Data Augmentation — Motivation & Transforms

Motivation

BaselineCNN already generalizes well, so augmentation is not mainly for overfitting.

We test whether the same model becomes more robust to pose, shifts, lighting and noise.

This addresses the limitation of training the baseline without augmentation.

Baseline CNN

Same configuration as previous experiment:

4 conv blocks + 3 FC layers, lr=1e-3, 20 epochs

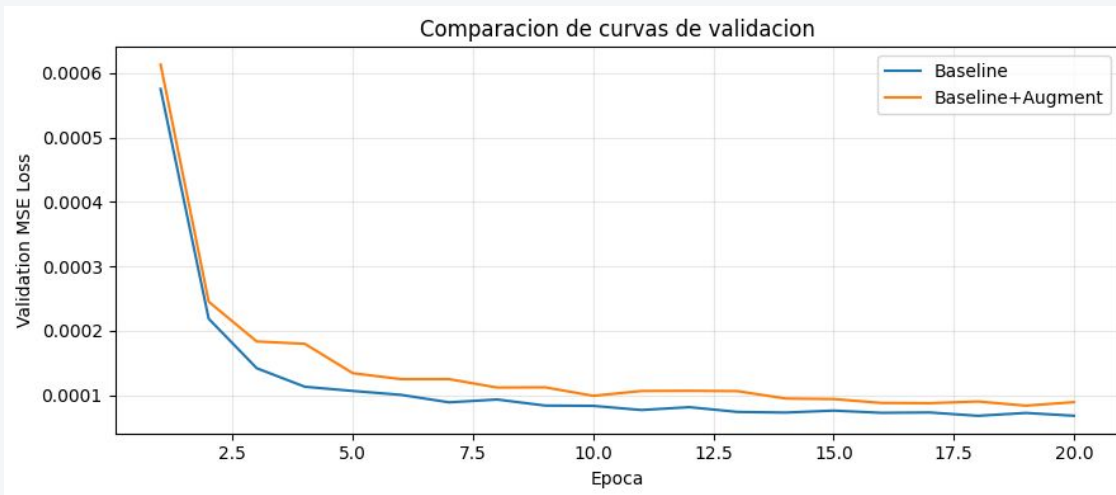
Data Augmentation Parameters

Parameter	Value
P_FLIP	0.5
P_ROTATE	0.5
P_SHIFT	0.5
P_BRIGHTNESS	0.5
P_NOISE	0.5
MAX_ANGLE_DEG	12.0
MAX_SHIFT_PX	6.0
BRIGHT_RANGE	(0.6, 1.4)
NOISE_STD	0.02

Data Augmentation — Quantitative Results

Validation Results Comparison

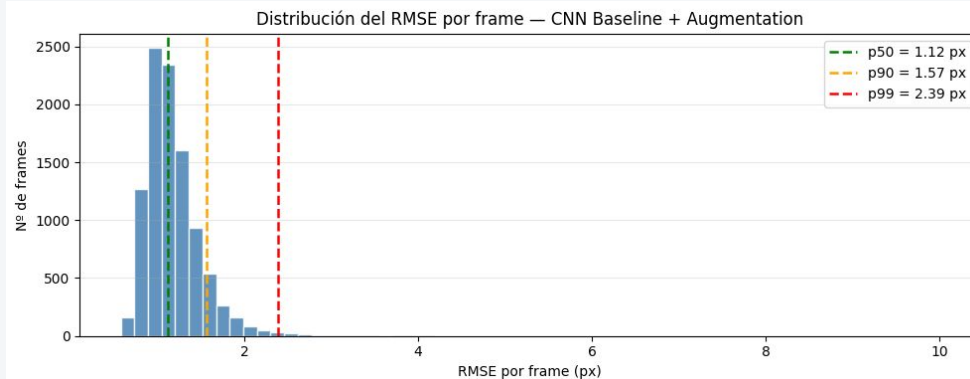
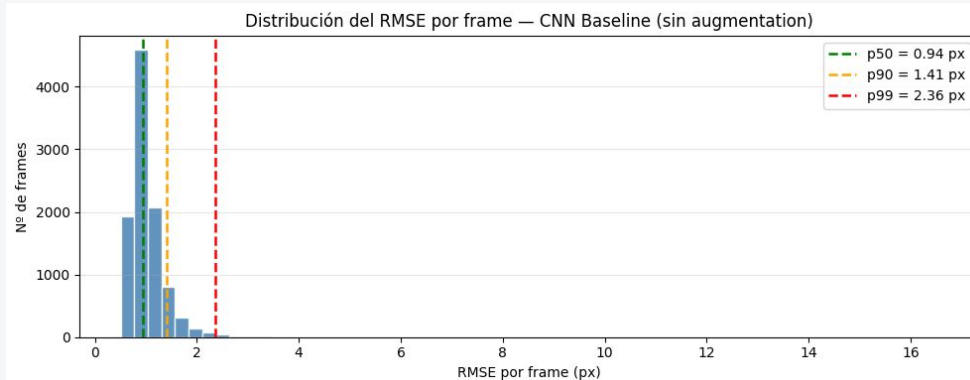
Model	Frames	LR	Best val MSE	RMSE (px)
Baseline CNN	50k	1e-3	0.000068	0.91 px
Baseline CNN + Data Augmentation	50k	1e-3	0.000084	1.05 px



Data Augmentation — Quantitative Results

RMSE PER FRAME DISTRIBUTION

1. Both distributions are right-skewed
2. Augmentation shifts the bulk slightly right
3. p99 stays almost flat $2.36 \rightarrow 2.39$ px
4. The extreme tail shrinks
5. Net effect: robustness over precision



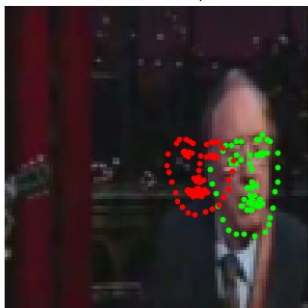
Data Augmentation — Qualitative Analysis

5 mejores (arriba) y 5 peores (abajo) frames por RMSE — CNN Baseline (sin augmentation)

MEJOR #1
RMSE = 0.51 px



PEOR #1
RMSE = 16.52 px



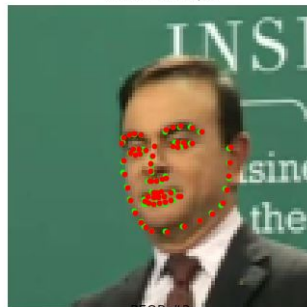
MEJOR #2
RMSE = 0.52 px



PEOR #2
RMSE = 13.10 px



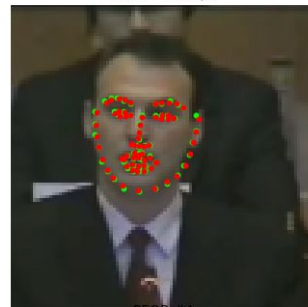
MEJOR #3
RMSE = 0.52 px



PEOR #3
RMSE = 8.94 px



MEJOR #4
RMSE = 0.52 px



PEOR #4
RMSE = 8.45 px



MEJOR #5
RMSE = 0.53 px



PEOR #5
RMSE = 7.63 px

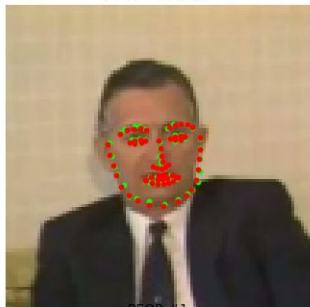


Real Predicho

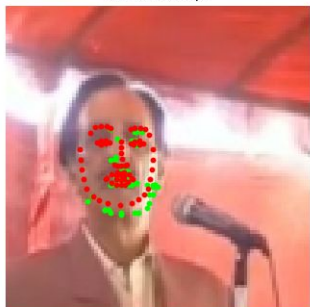
Data Augmentation — Qualitative Analysis

5 mejores (arriba) y 5 peores (abajo) frames por RMSE — CNN Baseline + Augmentation

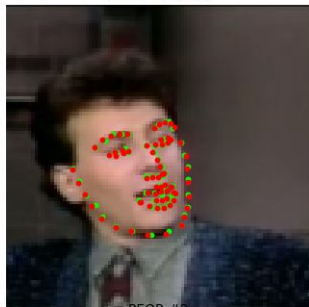
MEJOR #1
RMSE = 0.58 px



PEOR #1
RMSE = 9.99 px



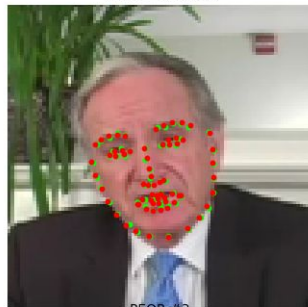
MEJOR #2
RMSE = 0.58 px



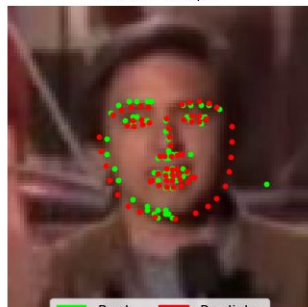
PEOR #2
RMSE = 9.57 px



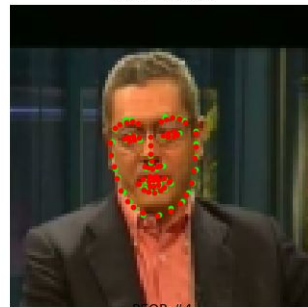
MEJOR #3
RMSE = 0.58 px



PEOR #3
RMSE = 7.75 px



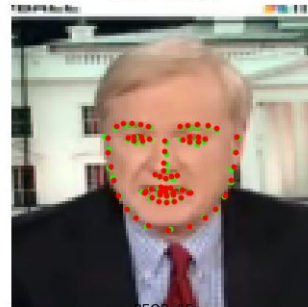
MEJOR #4
RMSE = 0.59 px



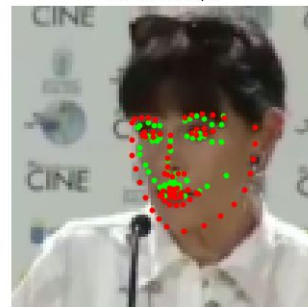
PEOR #4
RMSE = 6.51 px



MEJOR #5
RMSE = 0.61 px



PEOR #5
RMSE = 5.64 px



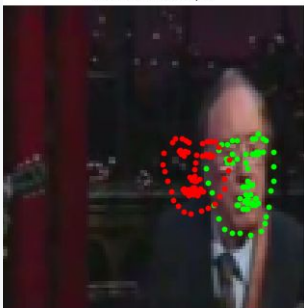
Real Predicho

Data Augmentation — Qualitative Analysis

WORST FRAME COMPARISON

BaselineCNN

RMSE = 16.52 px



RMSE = 13.10 px



RMSE = 8.94 px



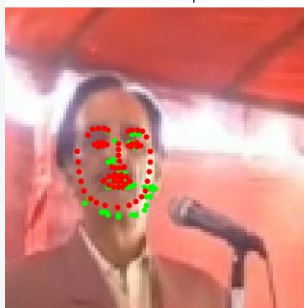
RMSE = 8.45 px



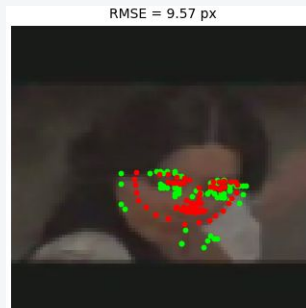
RMSE = 6.51 px



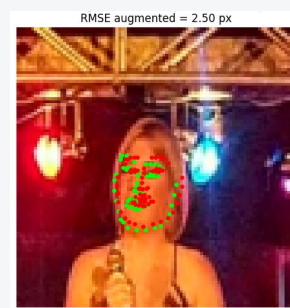
RMSE = 9.99 px



RMSE = 9.57 px



RMSE augmented = 2.50 px



BaselineCNN +
Augmentation

QAT (Quantization Aware Training)

PROBLEM

Neural networks are usually trained in 32-bit floating point (float 32), it is precise but:

- Uses a lot of memory
- Is computationally expensive

And we can't simply convert it to int8

SOLUTION

QAT solves this by simulating quantization during training.

The network learns how to work with low precision.

QAT (Quantization Aware Training): How does it work?

FORWARD PASS

Activations and weights are simulated as in int8.

```
# Quantization Stubs  
self.quant = quant.QuantStub()  
self.dequant = quant.DeQuantStub()
```

```
def forward(self, x):  
    x = self.quant(x)  
    x = self.features(x)  
    x = self.regressor(x)  
    x = self.dequant(x)  
    return x
```

Gradients are preserved in float32 for backpropagation.

BACKPROPAGATION

The loss and gradient computations remain in float32.

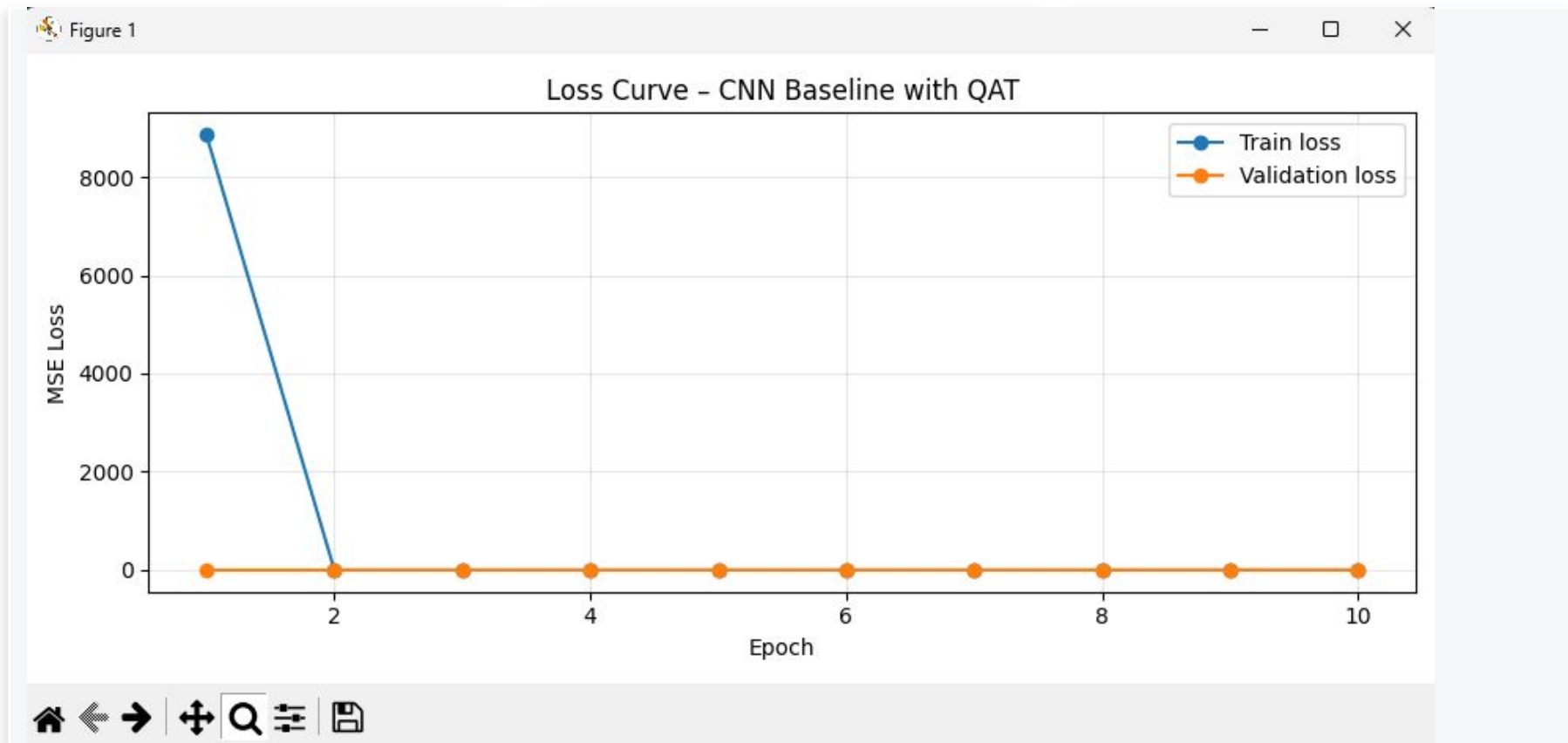
```
loss = criterion(model(imgs), kps)  
loss.backward()  
optimizer.step()
```

UPDATES

The network updates its float32 weights to minimize loss despite quantization errors.

```
optimizer.zero_grad()  
loss = criterion(model(imgs), kps)  
loss.backward()  
optimizer.step()
```

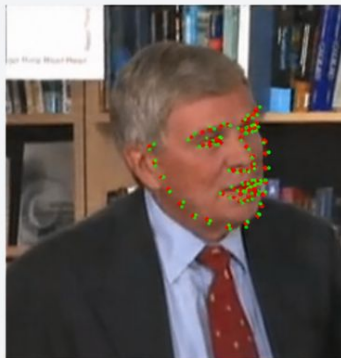
QAT (Quantization Aware Training): Results



QAT (Quantization Aware Training): Results

Best predictions

MEJOR #1
RMSE = 0.68 px



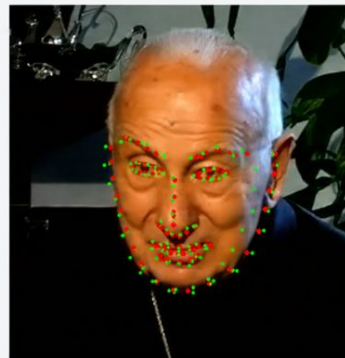
MEJOR #2
RMSE = 0.74 px



MEJOR #3
RMSE = 0.81 px



MEJOR #4
RMSE = 0.89 px



QAT (Quantization Aware Training): Results

Model	Frames	LR	Best val MSE	RMSE (px)
Baseline CNN	50k	1e-3	0.000068	0.91 px
Baseline CNN + Data Augmentation	50k	1e-3	0.000084	1.05 px
Baseline CNN + QAT	50k	1e-3	0.00092153	2.91 px

Conclusions

Key Findings

0.89 px RMSE

Baseline CNN achieves sub-pixel accuracy on 96×96 images

0.1% gap

val→test difference confirms no overfitting — model generalizes

lr matters

NaimishNet: 3.15 px (lr=1e-3) → 1.64 px (lr=1e-4) — 4× improvement

Simple > Complex

Well-tuned simple CNN outperforms complex model not adapted to dataset

Lower worst frames

Augmentation trades precision for robustness, but it is not a free win on this baseline.

Future Work

NaimishNet

Data augmentation (horizontal flip + brightness/contrast jitter).

NaimishNet

Transfer learning with ResNet50 pretrained on ImageNet.

NaimishNet

Tune Dropout rates specifically for this dataset.

Data augmentation

Report results over multiple seeds.

Data augmentation

Try offline augmentation.

References

[1] Facial Key Points Detection using Deep Convolutional Neural Network
https://github.com/mminamina/Transfer_Learning_NaimishNet---Keypoints_Detection

[2] Kaggle — Facial Keypoints Detection [Facial Keypoints Detection | Kaggle](#)

[3] YouTube Faces with Facial Keypoints: [YouTube Faces With Facial Keypoints](#)

[4] ishandutta0098/[GitHub - ishandutta0098/facial-keypoints-detection: Deep Learning Models for Facial Keypoints Detection : GitHub](#)

References

[5]

Data Augmentation for Facial Keypoint Detection

<https://www.kaggle.com/code/balraj98/data-augmentation-for-facial-keypoint-detection>

[6]

Facial Keypoints Detection with Post Training Quantization

https://colab.research.google.com/github/ishandutta0098/facial-keypoints-detection/blob/main/notebooks/post_training_quantization.ipynb